

Basic String Concepts in Python

1. What is a String?

- A string in Python is a sequence of characters enclosed within **single** (' '), **double** (" "), or **triple quotes** (''' ' or "" "" """).
- Strings are **immutable** and **ordered**. `s1 = 'Hello'`
- Ex.
 - `s2 = "Python"`
 - `s3 = '''Multi-line String'''`

2. Indexing

- Indexing **allows accessing characters** using their position.
- **Forward Indexing:**
 - `H e l l o`
`0 1 2 3 4`
 - `print("Hello"[0]) # H`
- **Reverse Indexing:**
 - `H e l l o`
`-5 -4 -3 -2 -1`
 - `print("Hello"[-1]) # o`

3. String Slicing

- **Syntax:** `string[start:end:step]`
 - `text = "Python"`
 - `print(text[0:3]) # Pyt`
 - `print(text[2:]) # thon`
 - `print(text[:3]) # Pyt`
 - `print(text[::-1]) # nohtyP (reverse)`

4. Reverse String

- `text = "Vishal"`
 - `print(text[::-1])` # lahsiv

5. Important Notes

- Strings are **immutable** (cannot change directly):
 - `text[0] = 'J'` # **Error**

➤ Valid Way to Modify String:

- `text = 'Python'`
`text = 'J' + text[1:]`
`print(text)` # Jython
-

Python String Methods

Core Methods with Examples and Cases

1. len()

It is a built-in function and Returns length of string

example:

- `print(len("hello")) # 5`
- `print(len("")) # 0 (empty string)`
- `print(len(" ")) # 2 (spaces count)`

2. count()

Counts occurrences of substring

example:

- `print("banana".count("a")) # 3`
- `print("banana".count("an")) # 2`
- `print("banana".count("z")) # 0`

3. title()

Converts first letter of each word to uppercase

example:

- `print("hello world".title()) # Hello World`

4. lower()

Converts all characters to lowercase

example:

- `print("HELLO".lower()) # hello`

5. upper()

Converts all characters to uppercase

example:

- `print("hello".upper()) # HELLO`

6. istitle()

Returns True if each word starts with uppercase

example:

- `print("Hello World".istitle()) # True`
- `print("hello world".istitle()) # False`

7. islower()

Returns True if all characters are lowercase

example:

- `print("hello".islower()) # True`
- `print("Hello".islower()) # False`

8. isupper()

Returns True if all characters are uppercase

example:

- `print("HELLO".isupper()) # True`
- `print("Hello".isupper()) # False`

9. find()

Finds first occurrence index, returns -1 if not found

example:

- `print("banana".find("a")) # 1`
- `print("banana".find("z")) # -1`

10. rfind()

Finds last occurrence index

example:

- `print("banana".rfind("a")) # 5`

11. index()

Similar to find(), raises error if not found

example:

- `print("banana".index("a")) # 1`
- `# print("banana".index("z")) # Error`

12. rindex()

Finds last occurrence index, raises error

example:

- `print("banana".rindex("a")) # 5`

Feature	find()	index()
Return value if substring is found	Returns index	Returns index
Return value if substring is not found	Returns -1	Raises error (ValueError)
Use case	Safer to use when unsure about substring	Use when substring is guaranteed to exist
Syntax	<code>string.find(substring)</code>	<code>string.index(substring)</code>

13. replace()

Replace substring in string

example:

- `print("hello world".replace("world", "Python")) # hello Python`

14. isalpha()

True if all chars are alphabets

example:

- `print("abc".isalpha()) # True`
- `print("abc123".isalpha()) # False`

15. isalnum()

True if alphanumeric only

example:

- `print("abc123".isalnum()) # True`
- `print("abc 123".isalnum()) # False`

16. isdecimal()

- Returns True only if all characters are **decimal digits (0–9)**.
- Does not consider superscripts, fractions, or symbols.

example:

- `print("123".isdecimal())` # True
- `print("12.3".isdecimal())` # False (contains .)
- `print("²".isdecimal())` # False (superscript)

17. isdigit()

- True **if all characters are digits**
- Returns True if all characters are digits, including Unicode digits like **superscripts (², ³)**.

example:

- `print("123".isdigit())` # True
- `print("12³".isdigit())` # True (³ is Unicode digit)
- `print("12.3".isdigit())` # False

18. isnumeric()

- Returns **True if all characters are numeric**, which includes:
- Decimal digits (0–9)
- Unicode digits (², ³)
- **Fractions**, e.g., ½ (Unicode character)
- **Roman numerals**, e.g., "VII"

example:

- `print("123".isnumeric())` # True
- `print("²".isnumeric())` # True
- `print("⅓".isnumeric())` # True (Unicode fraction)
- `print("12.3".isnumeric())` # False

Feature	isdecimal()	isdigit()	isnumeric()
Only 0–9 digits	✓	✓	✓
Unicode digits (² , ³)	✗	✓	✓
Fractions (¹ / ₃ , ¹ / ₂)	✗	✗	✓
Roman numerals (V)	✗	✗	✓
Decimal number with dot ("12.3")	✗	✗	✗
Negative number ("-12")	✗	✗	✗
Strictness level	Highest	Medium	Lowest

19. strip()

Removes leading and trailing spaces

example:

- `print(" hello ".strip()) # hello`

20. lstrip()

Removes only left-side spaces

example:

- `print(" hello".lstrip()) # hello`

21.rstrip()

Removes only right-side spaces

example:

- `print("hello ".rstrip()) # hello`

22. split()

Splits string into list by given substring default by space

example:

- `print("a,b,c".split(","))` # ['a', 'b', 'c']
- `print("2025-11-19".split("-"))` # ['2025', '11', '19']
- `print("one two three four".split(" ", 2))` # ['one', 'two', 'three four']
- `print("Python is awesome".split())` # ['Python', 'is', 'awesome']

23. format()

➤ The `format()` method is used to **insert values into a string** at the placeholders `{}`.

➤ **Syntax:**

- `string.format(value1, value2, ...)`

example :

- `print("My name is {} and I am {} years old.".format("Vishal", 20))`
My name is Vishal and I am 20 years old.
- `print("Programming in {1} is easier than {0}.".format("C", "Python"))`
Programming in Python is easier than C.
- `print("Hello, {name}! Your score is {score}.".format(name="Vishal", score=95))`
Hello, Vishal! Your score is 95.
- `print("Pi = {:.2f}".format(3.14159))` # Pi = 3.14
- `print("|{:>10}|{: ^10}|{:<10}|".format("Right", "Center", "Left"))`
| Right| Center |Left |